

# PDF Document Testing Corpus

Text · Tables · Images · Diagrams · Flowcharts

Version 1.0 | RAG & Document AI Pipeline Testing

Generated for benchmark evaluation purposes



# Table of Contents

---

| Section | Title                        | Page |
|---------|------------------------------|------|
| 1       | Introduction & Overview      | 3    |
| 2       | Data Tables                  | 4    |
| 3       | Charts & Visualisations      | 5    |
| 4       | System Architecture Diagram  | 7    |
| 5       | Query Processing Flowchart   | 8    |
| 6       | Embedding Similarity Heatmap | 9    |
| 7       | Technical Reference          | 10   |
| 8       | Glossary                     | 11   |

# 1. Introduction & Overview

This document serves as a comprehensive testing corpus for Document AI systems, PDF parsing pipelines, Retrieval-Augmented Generation (RAG) frameworks, and multimodal language models. It deliberately combines rich, heterogeneous content types — prose, structured tables, charts, architectural diagrams, and flowcharts — to stress-test extraction, understanding, and reasoning capabilities across a broad spectrum of document elements.

Retrieval-Augmented Generation (RAG) is a technique that enhances large language model responses by grounding them in retrieved external knowledge. The system embeds a user query into a high-dimensional vector space, performs approximate nearest-neighbour (ANN) search against a pre-indexed document store, and passes the top-k retrieved chunks to the LLM as context before generation. RAG reduces hallucinations, supports knowledge updates without retraining, and enables traceable source citations.

## 1.1 Key Evaluation Dimensions

A robust document AI pipeline must handle the following content modalities without degradation in extraction quality:

| # | Modality           | Test Element in This PDF | Failure Mode                   |
|---|--------------------|--------------------------|--------------------------------|
| 1 | Prose Text         | Sections 1, 7, 8         | Truncation, encoding errors    |
| 2 | Structured Tables  | Section 2                | Merged cells, misaligned cols  |
| 3 | Bar / Line Charts  | Section 3                | Alt-text absence, axis misread |
| 4 | Pie Chart          | Section 3                | Label occlusion, value errors  |
| 5 | Architecture Diag. | Section 4                | Arrow directionality loss      |
| 6 | Flowchart          | Section 5                | Decision branch confusion      |
| 7 | Heatmap            | Section 6                | Cell value extraction errors   |
| 8 | Code Blocks        | Section 7                | Whitespace / indent collapse   |

**Note:** All numerical values in this document are synthetic and designed for testing purposes only. They do not reflect real-world system performance.

## 2. Data Tables

### 2.1 RAG System Performance Comparison

Table 1 compares five RAG configurations across six standard benchmark metrics. Each configuration varies the embedding model, chunk size, and retrieval strategy to isolate the effect of individual pipeline components.

| System | Embed Model | Chunk Size | Faithfulness | Ans. Relevancy | Ctx Precision | Ctx Recall | RAGAS |
|--------|-------------|------------|--------------|----------------|---------------|------------|-------|
| RAG-A  | ada-002     | 256        | 0.91         | 0.87           | 0.83          | 0.79       | 0.85  |
| RAG-B  | ada-002     | 512        | 0.88         | 0.84           | 0.80          | 0.82       | 0.84  |
| RAG-C  | BGE-large   | 256        | 0.93         | 0.89           | 0.86          | 0.81       | 0.87  |
| RAG-D  | BGE-large   | 512        | 0.90         | 0.86           | 0.84          | 0.85       | 0.86  |
| RAG-E  | E5-mistral  | 128        | 0.85         | 0.81           | 0.77          | 0.74       | 0.79  |
| Broken | ada-002     | 256        | 0.61         | 0.54           | 0.48          | 0.42       | 0.51  |

Table 1 — RAG configuration benchmark results (synthetic data). Red row = broken corpus baseline.

### 2.2 Document Corpus Statistics

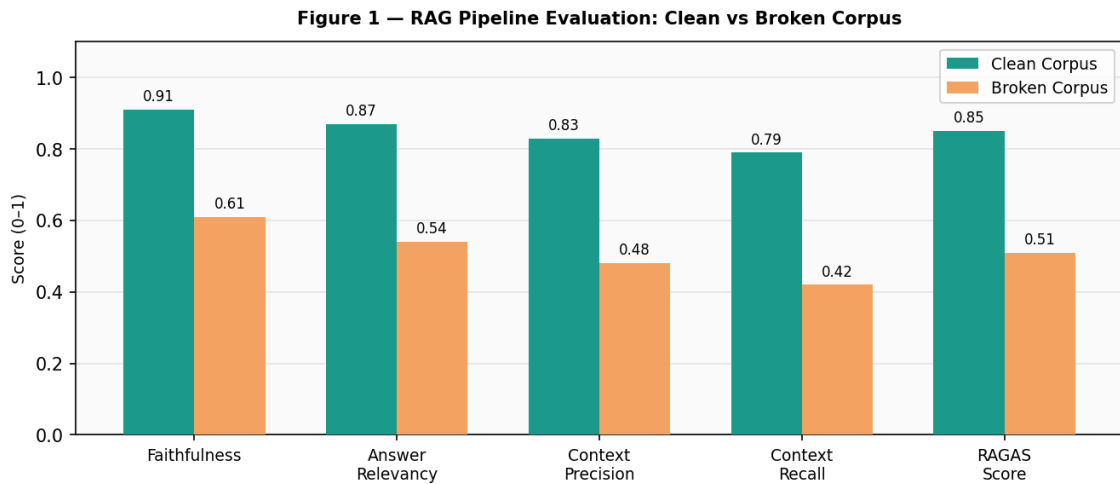
| Domain          | Doc Count | Avg Tokens/Doc | Avg Sentences | Top Entity Type |
|-----------------|-----------|----------------|---------------|-----------------|
| AI / ML         | 28        | 312            | 18            | Technical term  |
| Medicine        | 22        | 287            | 16            | Gene / protein  |
| History         | 18        | 341            | 20            | Person / date   |
| Climate Science | 15        | 298            | 17            | Measurement     |
| Economics       | 14        | 276            | 15            | Statistic       |
| Technology      | 12        | 265            | 14            | Acronym         |
| Astronomy       | 10        | 318            | 18            | Celestial body  |
| Nutrition       | 9         | 244            | 13            | Nutrient        |
| TOTAL           | 128       | 293 (avg)      | 16 (avg)      | —               |

Table 2 — Corpus domain statistics (synthetic).

### 3. Charts & Visualisations

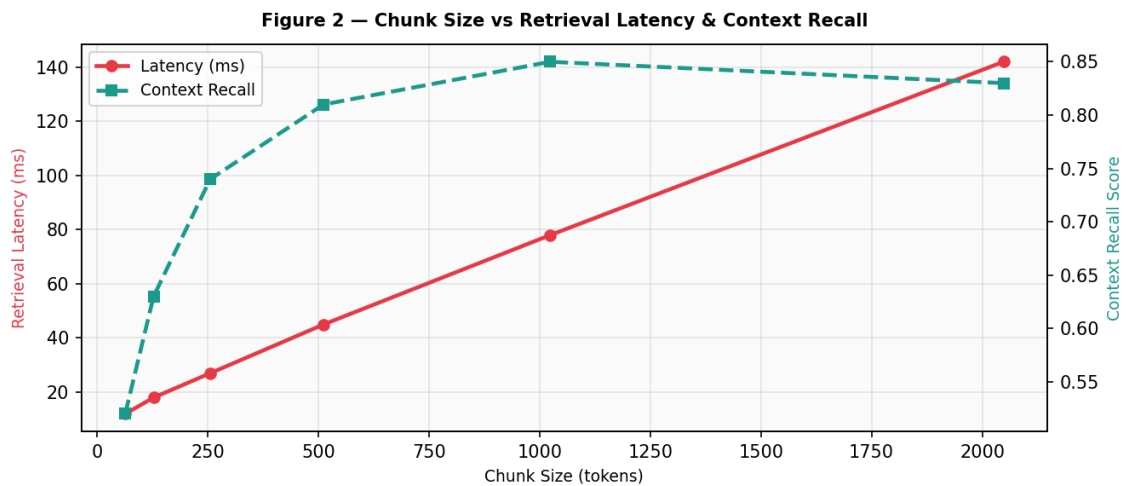
#### 3.1 RAG Metric Scores — Clean vs Broken Corpus

Figure 1 shows that corpus data quality is the single largest driver of pipeline performance. Broken corpus conditions degrade RAGAS scores by an average of 39.5% relative to the clean baseline.



#### 3.2 Chunk Size vs Retrieval Latency & Context Recall

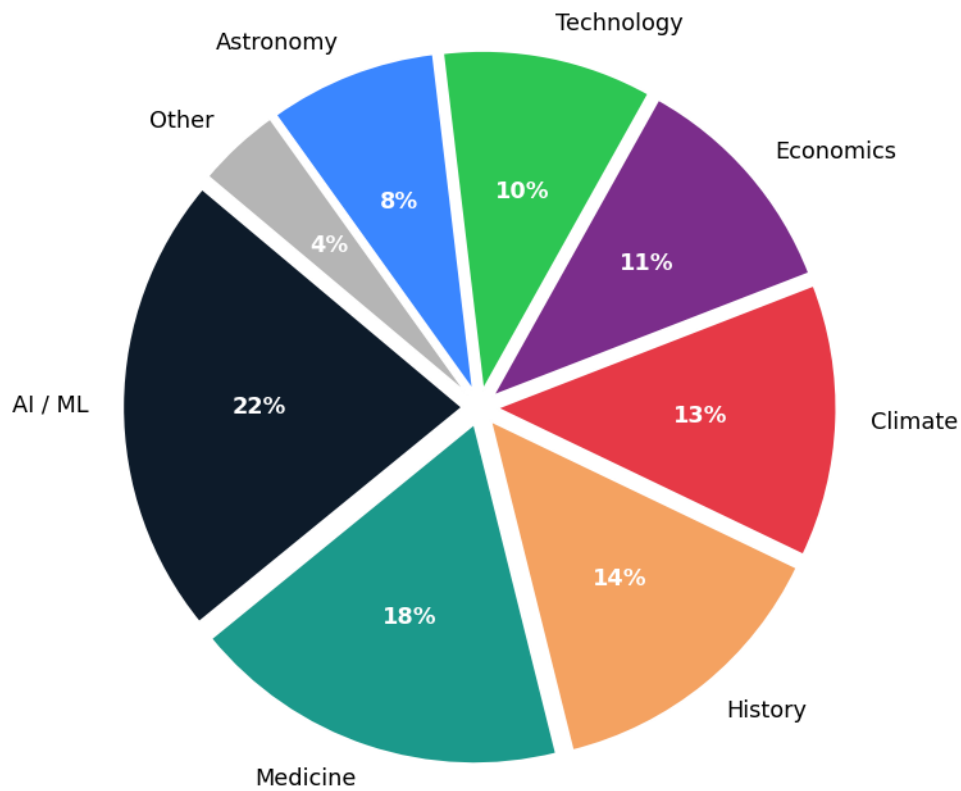
Figure 2 illustrates the tension between retrieval speed and recall quality. While larger chunks (1024+ tokens) improve context recall, they cause a near-exponential increase in retrieval latency — a key operational trade-off.



### 3.3 Corpus Domain Distribution

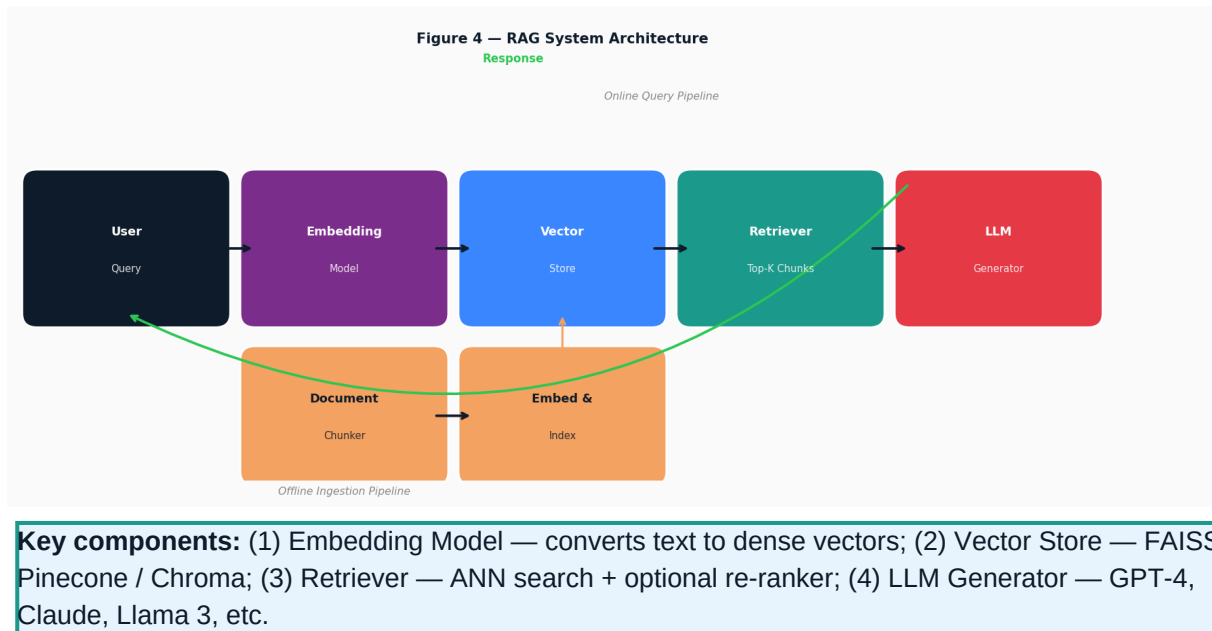
Figure 3 shows the domain breakdown of the test corpus by document count. AI/ML dominates at 22%, followed by Medicine (18%) and History (14%). A balanced distribution is critical for fair cross-domain retrieval evaluation.

**Figure 3 — Corpus Domain Distribution**



## 4. System Architecture Diagram

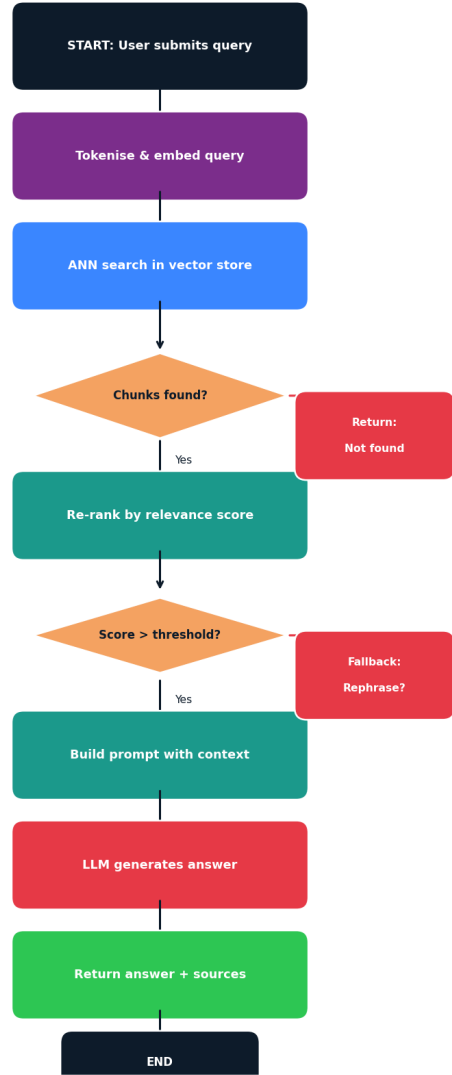
Figure 4 presents the end-to-end RAG system architecture. The pipeline has two distinct phases: an **offline ingestion pipeline** (bottom row) that chunks, embeds, and indexes documents, and an **online query pipeline** (top row) that embeds the user query, retrieves relevant chunks, and passes them to the LLM generator. The green arrow represents the response returned to the user.



# 5. Query Processing Flowchart

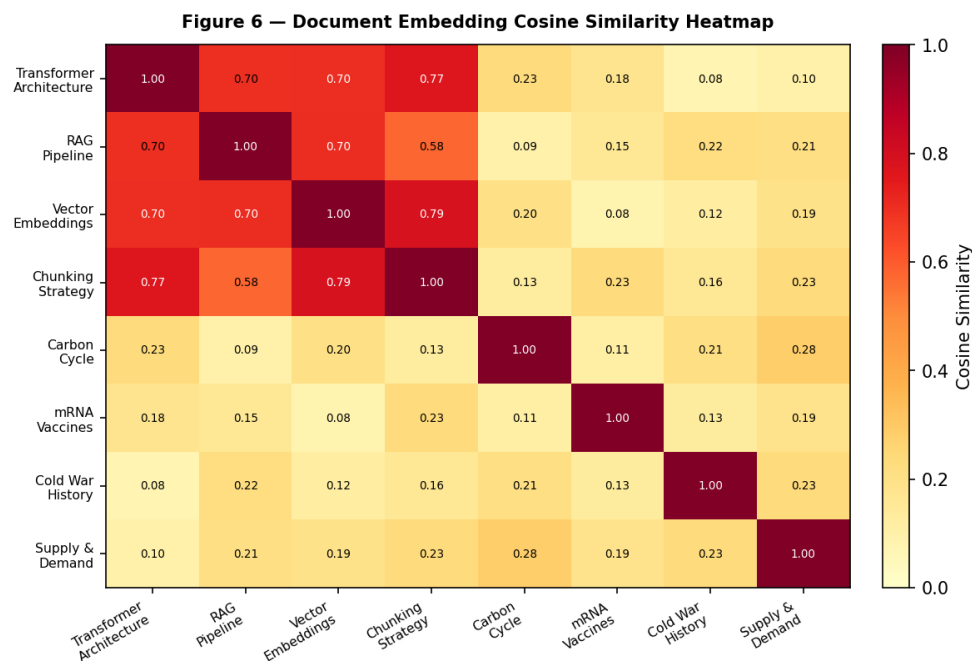
Figure 5 details the step-by-step decision logic for processing a user query through the RAG pipeline. Two decision diamonds handle edge cases: (a) no chunks found — return a not-found response; (b) chunks found but below relevance threshold — trigger rephrasing fallback.

Figure 5 — RAG Query Processing Flowchart



## 6. Embedding Similarity Heatmap

Figure 6 shows pairwise cosine similarity scores between eight document embeddings from the test corpus. Documents within the AI/ML cluster (Transformer Architecture, RAG Pipeline, Vector Embeddings, Chunking Strategy) show high intra-cluster similarity (0.55–0.85), while cross-domain pairs (e.g., Carbon Cycle vs Cold War History) score below 0.30.



This heatmap is used to benchmark whether a RAG system's retrieval correctly surfaces semantically related documents and avoids retrieving low-similarity cross-domain noise.

## 7. Technical Reference

### 7.1 Embedding Similarity Formula

Cosine similarity between two document vectors A and B is defined as:

$$\text{cosine\_similarity}(A, B) = (A \cdot B) / (||A|| \times ||B||)$$

Where  $A \cdot B$  is the dot product and  $||A||$  is the L2 norm. Scores range from -1 (opposite) to +1 (identical); typical semantic similarity thresholds are 0.7–0.9 for high relevance.

### 7.2 Sample Python: Embedding + Retrieval

```
from sentence_transformers import SentenceTransformer import faiss, numpy as np model =
SentenceTransformer('BAAI/bge-large-en-v1.5') docs = ['Transformers use self-attention.',
'RAG reduces hallucinations.'] embeddings = model.encode(docs, normalize_embeddings=True)
index = faiss.IndexFlatIP(embeddings.shape[1]) index.add(embeddings.astype('float32'))
query_vec = model.encode(['What is self-attention?'], normalize_embeddings=True) scores,
ids = index.search(query_vec.astype('float32'), k=2) print(scores, ids) # Returns top-2
matches
```

### 7.3 Evaluation Thresholds

| Metric            | Excellent | Good      | Acceptable | Poor   |
|-------------------|-----------|-----------|------------|--------|
| Faithfulness      | > 0.90    | 0.75–0.90 | 0.60–0.75  | < 0.60 |
| Answer Relevancy  | > 0.88    | 0.72–0.88 | 0.55–0.72  | < 0.55 |
| Context Precision | > 0.85    | 0.70–0.85 | 0.55–0.70  | < 0.55 |
| Context Recall    | > 0.82    | 0.67–0.82 | 0.50–0.67  | < 0.50 |
| RAGAS Score       | > 0.87    | 0.72–0.87 | 0.55–0.72  | < 0.55 |

Table 3 — RAG metric quality thresholds.

## 8. Glossary

---

### ***ANN***

Approximate Nearest Neighbour — a fast search algorithm that finds vectors close to a query vector in high-dimensional space.

### ***Chunking***

The process of splitting a long document into smaller pieces (chunks) before embedding and indexing.

### ***Cosine Similarity***

A measure of similarity between two non-zero vectors, computed as the cosine of the angle between them.

### ***Embedding***

A dense numerical vector representation of text or other data in a continuous high-dimensional space.

### ***Faithfulness***

A RAG metric measuring whether a generated answer is fully grounded in the retrieved context without added hallucinations.

### ***FAISS***

Facebook AI Similarity Search — an open-source library for efficient similarity search over dense vectors.

### ***Hallucination***

When an LLM generates plausible-sounding but factually incorrect information not grounded in retrieved context.

### ***HNSW***

Hierarchical Navigable Small World — a graph-based ANN index that offers high recall with low query latency.

### ***LLM***

Large Language Model — a neural network trained on vast text corpora to generate, summarise, and reason over natural language.

### ***RAGAS***

Retrieval-Augmented Generation Assessment — a framework for automated evaluation of RAG pipelines across four key metrics.

### ***RAG***

Retrieval-Augmented Generation — a technique that combines information retrieval with generative language models.

### ***Re-ranking***

A post-retrieval step that reorders candidate chunks by a cross-encoder relevance model before passing to the LLM.

### ***Top-k***

The k most relevant document chunks returned by the retriever for a given query vector.

### ***Vector Store***

A database optimised for storing and querying high-dimensional embedding vectors (e.g., Pinecone, Weaviate, Chroma, Qdrant).